Assignment 4: Introduction to MPI

1. Introduction to MPI

Message Passing Interface (MPI) is a standardized and portable message-passing system designed to function on parallel computing architectures. MPI is widely used in high-performance computing for writing parallel programs that can run on distributed memory systems.

Key Features of MPI:

- Scalability: Works from dual-core to supercomputers
- Portability: Runs on various platforms (Linux, Windows, macOS)
- Performance: Optimized for high-speed communication
- Standardization: Well-defined API with multiple implementations (MPICH, OpenMPI)

Common Use Cases:

- Scientific simulations (weather, climate modeling)
- Computational fluid dynamics
- Machine learning on distributed systems
- Large-scale data processing
- Parallel numerical algorithms

2. Setting Up MPI Environment

Installation on Linux (Ubuntu/Debian):

```
$ sudo apt-get update
$ sudo apt-get install mpich
$ mpicc --version
```

Compiling and Running MPI Programs:

To compile an MPI program written in C:

```
$ mpicc -o program_name program_name.c
```

To run the compiled program with 4 processes:

```
$ mpirun -np 4 ./program_name
```

3. Basic MPI Concepts

3.1 Processes and Ranks

In MPI, each instance of your program is called a process. Each process has a unique identifier called a rank (starting from 0). The total number of processes is called the size of the communicator.

3.2 Communicators

MPI_COMM_WORLD is the default communicator that includes all processes. It's the most commonly used communicator in MPI programs.

3.3 Essential MPI Functions

Function	Description
MPI_Init(&argc, &argv)	Initialize MPI environment
MPI_Comm_size(comm, &size)	Get number of processes
MPI_Comm_rank(comm, &rank)	Get process rank
MPI_Send(buf, count, type, dest, tag, comm)	Send message to another process
MPI_Recv(buf, count, type, src, tag, comm, &status)	Receive message from another process
MPI_Bcast(buf, count, type, root, comm)	Broadcast message to all processes
MPI_Reduce(sendbuf, recvbuf, count, type, op, root, comm)	Reduce values from all processes
MPI_Finalize()	Clean up MPI environment

4. Example Programs with DIY Runs

4.1 Hello World Program

Objective: Learn basic MPI initialization and process identification.

Program Code:

```
#include <mpi.h>
#include <stdio.h>
```

```c
int main(int argc, char** argv) {
    // Initialize MPI environment
    MPI_Init(&argc, &argv);

    // Get total number of processes
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // Get rank of current process
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // Get processor name
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);

    // Print hello message
    printf("Hello from processor %s, rank %d out of %d processors\n",
        processor_name, world_rank, world_size);

    // Finalize MPI environment
    MPI_Finalize();
    return 0;
}
```

DIY Run Instructions:

1. Save the code as hello_mpi.c

2. Compile: mpicc -o hello_mpi hello_mpi.c

3. Run with 4 processes: mpirun -np 4 ./hello_mpi

Expected Output:

```
Hello from processor localhost, rank 0 out of 4 processors
Hello from processor localhost, rank 1 out of 4 processors
Hello from processor localhost, rank 2 out of 4 processors
Hello from processor localhost, rank 3 out of 4 processors
```

Key Concepts:

MPI_Init() must be called before any other MPI function

Each process executes the same code independently

Rank identifies each process uniquely (0 to size-1)

MPI_Finalize() must be called before program termination

4.2 Point-to-Point Communication

Objective: Learn how to send and receive messages between two specific processes.

Program Code:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    int number;
    if (world_rank == 0) {
        // Process 0 sends a number to process 1
        number = 42;
        printf("Process 0 sending number %d to process 1\n", number);
        MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    else if (world_rank == 1) {
        // Process 1 receives the number from process 0
        MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
        printf("Process 1 received number %d from process 0\n", number);
    }

    MPI_Finalize();
    return 0;
}
```

DIY Run Instructions:

1. Save as send_recv.c

2. Compile: mpicc -o send_recv send_recv.c

3. Run: mpirun -np 2 ./send_recv

Expected Output:

Process 0 sending number 42 to process 1

Process 1 received number 42 from process 0

Understanding MPI_Send and MPI_Recv:

Parameter	**Description**
buf	Pointer to data buffer

count	Number of elements to send/receive
datatype	MPI datatype (MPI_INT, MPI_FLOAT, etc.)
dest/source	Rank of destination/source process
tag	Message identifier (for matching)
comm	Communicator (usually MPI_COMM_WORLD)

4.3 Collective Communication - Broadcast

Objective: Learn how to broadcast data from one process to all other processes.

Program Code:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    int data;

    if (world_rank == 0) {
        data = 100;
        printf("Process 0 broadcasting value: %d\n", data);
    }

    // Broadcast data from process 0 to all processes
    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);

    if (world_rank != 0) {
        printf("Process %d received broadcasted value: %d\n",
            world_rank, data);
    }

    MPI_Finalize();
    return 0;
}
```

DIY Run Instructions:

1. Save as broadcast.c

2. Compile: mpicc -o broadcast broadcast.c

3. Run: mpirun -np 4 ./broadcast

Expected Output:

Process 0 broadcasting value: 100

Process 1 received broadcasted value: 100

Process 2 received broadcasted value: 100

Process 3 received broadcasted value: 100

4.4 Collective Communication - Reduction

Objective: Learn how to perform reduction operations (sum, max, min) across all processes.

Program Code:

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // Each process has its own local value
    int local_value = world_rank + 1;
    printf("Process %d has local value: %d\n", world_rank, local_value);

    // Sum all local values and store result in process 0
    int global_sum;
    MPI_Reduce(&local_value, &global_sum, 1, MPI_INT,
        MPI_SUM, 0, MPI_COMM_WORLD);

    if (world_rank == 0) {
        printf("\nGlobal sum: %d\n", global_sum);
        printf("Expected: %d\n", (world_size * (world_size + 1)) / 2);
    }

    MPI_Finalize();
    return 0;
}
```

DIY Run Instructions:

1. Save as reduce_sum.c

2. Compile: mpicc -o reduce_sum reduce_sum.c

3. Run: mpirun -np 4 ./reduce_sum

Expected Output:

```
Process 0 has local value: 1

Process 1 has local value: 2

Process 2 has local value: 3

Process 3 has local value: 4


Global sum: 10

Expected: 10
```

Common MPI Reduction Operations:

Operation	Description
MPI_SUM	Sum of all values
MPI_PROD	Product of all values
MPI_MAX	Maximum value
MPI_MIN	Minimum value
MPI_LAND	Logical AND
MPI_LOR	Logical OR

4.5 Parallel Matrix Multiplication

Objective: Implement parallel matrix-vector multiplication using MPI.

Program Code:

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 4  // Matrix size

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int rows_per_proc = N / size;
    // Full matrix and vector (only on process 0)
    int matrix[N][N];
    int vector[N];
    int result[N];
    // Local data for each process
    int local_matrix[rows_per_proc][N];
    int local_result[rows_per_proc];
```

```
    if (rank == 0) {
        // Initialize matrix and vector
        printf("Matrix:\n");
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                matrix[i][j] = i + j;
                printf("%d ", matrix[i][j]);
            }
            printf("\n");
        }
        printf("\nVector:\n");
        for (int i = 0; i < N; i++) {
            vector[i] = i;
            printf("%d ", vector[i]);
        }
        printf("\n\n");
    }
    // Scatter matrix rows to all processes
    MPI_Scatter(matrix, rows_per_proc * N, MPI_INT,
        local_matrix, rows_per_proc * N, MPI_INT,
        0, MPI_COMM_WORLD);
    // Broadcast vector to all processes
    MPI_Bcast(vector, N, MPI_INT, 0, MPI_COMM_WORLD);
    // Each process computes its portion
    for (int i = 0; i < rows_per_proc; i++) {
        local_result[i] = 0;
        for (int j = 0; j < N; j++) {
            local_result[i] += local_matrix[i][j] * vector[j];
        }
    }
    // Gather results back to process 0
    MPI_Gather(local_result, rows_per_proc, MPI_INT,
        result, rows_per_proc, MPI_INT,
        0, MPI_COMM_WORLD);
    if (rank == 0) {
        printf("Result:\n");
        for (int i = 0; i < N; i++) {
            printf("%d ", result[i]);
        }
        printf("\n");
    }
    MPI_Finalize();
    return 0;
}
```

DIY Run Instructions:

1. Save as matrix_mult.c

2. Compile: mpicc -o matrix_mult matrix_mult.c

3. Run: mpirun -np 4 ./matrix_mult

Key Concepts:

MPI_Scatter distributes data from root to all processes

MPI_Gather collects data from all processes to root

Each process works on its portion independently

Communication overhead must be considered for performance

6 Performance Measurement in MPI

Measuring performance is crucial for understanding the efficiency of parallel programs. MPI provides built-in timing functions, and we can calculate important metrics like speedup and efficiency.

6.1 MPI Timing Functions

MPI provides high-resolution wall-clock time measurement:

Function	Description
MPI_Wtime()	Returns elapsed wall-clock time in seconds (double precision)
MPI_Wtick()	Returns resolution of MPI_Wtime in seconds

Basic Timing Example:

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    // Start timing
    double start_time = MPI_Wtime();
    // Your computation here
    long long sum = 0;
    for (long long i = 0; i < 1000000000; i++) {
        sum += i;
    }
    // End timing
    double end_time = MPI_Wtime();
    printf("Process %d: Execution time = %f seconds\n",
        rank, end_time - start_time);
    MPI_Finalize();
    return 0;
}
```

Important Notes:

MPI_Wtime() returns wall-clock time (real time), not CPU time

Time is measured in seconds as a double-precision floating point

Always use MPI_Wtime() for parallel programs, not clock()

Synchronize processes with MPI_Barrier before timing critical sections

5.2 Performance Metrics

Compute: Execution Time (T_p),Speedup (S_p), Efficiency (E_p), Scalability, Communication Time and Computation Time.

Metric	**Formula & Description**
Execution Time (T_p)	Time taken with *p* processes
Speedup (S_p)	$S_p = T_1 / T_p$ (How much faster with *p* processes)
Efficiency (E_p)	$E_p = S_p / p = T_1 / (p \times T_p)$ (0 to 1, higher is better)
Scalability	How performance changes as problem size or processes increase
Communication Time	Time spent in communication operations
Computation Time	Time spent in actual computation

DIY Performance Testing:

1. Compile: mpicc -o eg1 eg1.c -O3
2. Run with 1 process: mpirun -np 1 ./eg1
3. Run with 2 processes: mpirun -np 2 ./eg1
4. Run with 4 processes: mpirun -np 4 ./eg1
5. Run with 8 processes: mpirun -np 8 ./eg1

Analysis Tasks:

Record execution times for 1, 2, 4, and 8 processes

Calculate speedup: $S_p = T_1 / T_p$

Calculate efficiency: $E_p = S_p / p$

Plot speedup vs number of processes

Identify the communication overhead

Determine optimal number of processes for problem

Sample Results Table:

Processes (p)	Time (T_p)	Speedup (S_p)	Efficiency (E_p)	Comm %
1	T_1 sec	1.00	100%	
2	T_2 sec	T_1/T_2	$S_2/2$	
4	T_4 sec	T_1/T_4	$S_4/4$	
8	T_8 sec	T_1/T_8	$S_8/8$	

6. Laboratory Exercises

Complete the following exercises to strengthen your MPI programming skills. For each exercise, write the complete program, test it with different numbers of processes, and analyze the results.

Exercise 1: Ring Communication
Problem: Write an MPI program where each process sends a message to the next process in a ring topology. Process 0 sends to Process 1, Process 1 sends to Process 2, and the last process sends back to Process 0.
Requirements:
Start with an initial value of 100 in Process 0
Each process adds its rank to the value before sending
Print the value at each process
Final value should return to Process 0
Hint: Use modulo operator to wrap around: next_rank = (rank + 1) % size
Test with: mpirun -np 4 ./ring_comm

Exercise 2: Parallel Array Sum
Problem: Write an MPI program to compute the sum of an array in parallel. Divide the array among processes, compute local sums, and combine them.
Requirements:
Create an array of size 100 with values 1 to 100 in Process 0
Distribute array portions using MPI_Scatter
Each process computes sum of its portion
Use MPI_Reduce to compute global sum
Verify result: sum should be 5050
Bonus: Also compute and print the average value.

Test with: mpirun -np 4 ./array_sum

Exercise 3: Finding Maximum and Minimum

Problem: Write an MPI program where each process generates random numbers, and the program finds the global maximum and minimum values across all processes.

Requirements:

Each process generates 10 random numbers (0-1000)
Each process finds its local maximum and minimum
Use MPI_Reduce with MPI_MAX and MPI_MIN
Process 0 prints the global maximum and minimum
Also print which process had these values (use MPI_MAXLOC/MPI_MINLOC)

Hint: For MPI_MAXLOC, use a struct with value and rank fields.

Exercise 4: Parallel Dot Product

Problem: Compute the dot product of two vectors in parallel.

Requirements:

Vector A = [1, 2, 3, 4, 5, 6, 7, 8]
Vector B = [8, 7, 6, 5, 4, 3, 2, 1]
Distribute vectors using MPI_Scatter
Each process computes partial dot product
Use MPI_Reduce to sum partial products
Expected result: 120